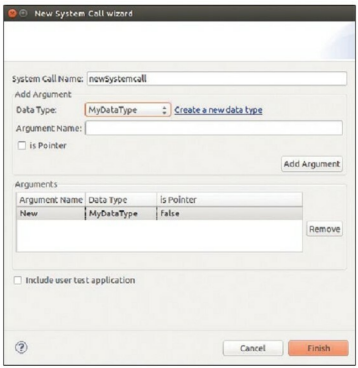


Figure 2: Add system call

using this plugin is that the remote system's *rootfs*, either running through QEMU or in the local network, will be mounted onto the IDE as a view. So transferring files to remote systems becomes easy, and the remote system shell can be viewed as a separate console in the LinK+ IDE. The complete procedure to connect with the remote systems is given in the user manual.

## Linux kernel debugging using QEMU

Debugging the Linux kernel image is possible with the Eclipse CDT debugger and the QEMU emulator. Avail the debugging option by right-clicking on the respective project under *Project Explorer*, go to *LinK to-> Emulation -> Debug* with QEMU and follow the steps as mentioned in the user manual.

## A project to create a Linux kernel system call

The LinK+ IDE has a wizard for adding new system calls to the kernel source. This can be done by right-clicking on the respective kernel project in the *Project Explorer* view, before going to *LinK to -> System Call -> Add System Call*. Clicking on *Add System Call* will open a wizard as shown in Figure 2.

Enter the name of the system call, then select the existing data type and click the *Add Argument* button. You can also create a new data type by clicking the *Create a new data type* link. If you want to test your system call with a system call application, enable the *Include User Test* application. It will create a user space application for system call testing. Finally, rebuild the project to make changes effective.

## System call testing with QEMU

There are just two steps to test how the system call works when using QEMU. Run the newly added system call kernel

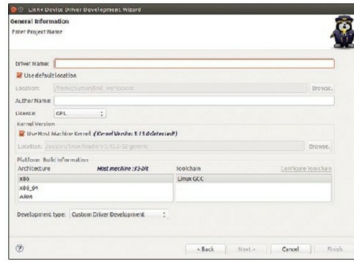


Figure 3: Toolchain list

image with QEMU and then copy the system call user application to the running QEMU. To do the above, go to *LinK to -> System Call*.

## A Linux device driver development project

Linux device driver projects are created by selecting *File->New->Project*. Then go to *Linux Kernel Development (Link+)* and select *Device Driver Project*.

This will open a wizard that contains three pages—one covering general information, one on the kernel's features and the page on driver information. You can finish the project at any point in time.

In the general information page, we need to fill the fields like author's name**,** license type, kernel version and, finally, select the architecture and the toolchain required for it from the *Toolchain* list as shown in Figure 3.

In the *Development type* dropdown box, there are two options: *Custom Driver Development* and *Typical Driver Development*. In the first option, select what you require but in the latter development type, a lot of features are automatically integrated.

Click on *Next* for other *Kernel Development Features* or *Finish* for *Basic module programming***.** If you choose the *Custom Driver Development* type, follow the procedure shown below.

## Advanced kernel module program and a project to create device drivers

The Linux kernel features' page is shown in Figure 4 and contains the kernel features like:

- Module *PARAM*
- Delayed Works ->Kernel Timer, Tasklet, Work Queue, Shared Queue
- Synchronization->Semaphore, Spinlock and other options
- Kernel Data Types
- Debugging Mechanisms -> probing, proc, sys attributes

All the features are optional. So you can select any feature based on requirements. On selecting a feature, the left pane